TRINEXIS: An Alert-Aware, Interval-Throttled Open-Vocabulary Webcam System with Language-Configured Speech

Yug G. Lakhani[1]
[1]Department of Computer Engineering, Gyanmanjari Innovative University, Bhavnagar, Gujarat, India
Corresponding author: Yug G. Lakhani, yglakhani@gmiu.edu.in
ORCID: https://orcid.org/0009-0004-2930-4953

Abstract— Open-vocabulary detectors can be configured with text prompts, but an interactive prototype also needs predictable control flow, bounded speech, and an explicit account of what has been evaluated. TRINEXIS is a Flask-based webcam system that couples an interval-throttled YOLO-World detector, a browser dashboard, language-configured speech output, session telemetry, and alert-aware narration. The system ranks user-designated alert objects before choosing the short spoken shortlist, uses wall-clock heatmap decay, reports a unitless relative distance index, and exposes configurable camera, security, and runtime settings. A deterministic synthetic regression check confirms that alert-first ranking preserves the configured alert object in cases where confidence-first shortlist truncation can omit it; the omission rate expected from the synthetic rank model shows that the check validates an ordering invariant rather than real-world detection behavior. The principal limitation is that no labeled detection dataset, hardware timing campaign, known-distance calibration, translated-template rating, or consented user study was conducted. This technical report therefore documents the system and its software verification without claiming detection quality, latency, distance error, multilingual comprehension, or assistive effectiveness.

Keywords / Index Terms— open-vocabulary detection; alert-aware narration; research software; speech interface; interval throttling; technical report

I. INTRODUCTION

A. Motivation

Object detection predicts bounding boxes and semantic labels for objects in an image. The YOLO family formulated detection as a single regression problem [1], and COCO-style benchmarks standardized evaluation over a fixed vocabulary of 80 released categories [2]. Vision-language pre-training broadened the interface: CLIP enabled zero-shot transfer from natural-language supervision [3], Grounding DINO fused language and vision for open-set detection [4], and YOLO-World introduced a prompt-then-detect paradigm [5]. Ultralytics exposes that family through a set_classes interface [6]. These advances make it feasible to build interactive applications around user-selected object names.

TRINEXIS is a Flask-based webcam application that combines an interval-throttled YOLO-World detector, a browser dashboard, language-configured speech output, detection logs, a heatmap, and session telemetry. Its maintained backend, app23.py, uses Flask control routes [7], OpenCV capture [8], Ultralytics YOLO [15], and gTTS speech [9]. The design question is how to turn a configurable open-vocabulary detector into a bounded, testable interaction loop with predictable alert handling.

B. Contribution

The system's focus is alert-aware, language-configured scene narration in an interval-throttled open-vocabulary webcam pipeline. User-designated alert objects are ranked before the spoken shortlist is truncated. This ordering is isolated in a dependency-free policy module, verified by a seeded synthetic regression check, and kept separate from empirical detection or user-benefit claims.

- A documented Flask/YOLO-World application with AGPL-3.0 licensing [24], pinned requirements, configurable camera/security settings, and vector figures.

- An alert-aware speech selector that ranks alerts before shortlist truncation, with a deterministic ordering-invariant check.
- A tagged release [23], CITATION.cff, codemeta.json, a weights download/checksum script, and a short software-verification appendix.

C. Scope and Limitations

This manuscript is a technical report for arXiv or an institutional repository; no journal or conference was selected. No labeled detection dataset, hardware timing campaign, known-distance calibration, translated-template rating, or consented user study was conducted; therefore no detection-quality, latency, distance-error, robustness, multilingual-comprehension, assistive-effectiveness, or user-study result is claimed. The generic single-column layout and numeric citations are fallbacks; any later journal or conference submission requires that venue's current Guide for Authors and official template.

II. RELATED WORK / LITERATURE REVIEW

A. Detection and Open-Vocabulary Models

The original YOLO paper evaluated PASCAL VOC 2007/2012 and artwork-domain generalization on the Picasso and People-Art datasets; it did not evaluate on COCO [1]. COCO later supplied a fixed-vocabulary benchmark [2]. CLIP [3], Grounding DINO [4], YOLO-World [5], OWL-ViT [11], GLIP [12], Detic [13], and YOLOE [14] represent successive approaches to language-conditioned or open-vocabulary recognition. Their published results were obtained under distinct datasets, splits, prompts, hardware, and protocols, so they are not treated here as comparative baselines for TRINEXIS. Table I summarizes scope and non-comparability.

TABLE I. VERIFIED RELATED WORK AND NON-COMPARABILITY

Source	Contribution	Verified evaluation context	Relevance and limitation
Redmon et al. [1], 2016	One-stage YOLO detector	PASCAL VOC 2007/2012; Picasso and People-Art artwork generalization	Established real-time closed-set detection; no COCO evaluation in that paper and a different hardware/protocol context
Lin et al. [2], 2014	Microsoft COCO benchmark	Approximately 328,000 images, 2.5 million labeled instances, 80 released categories	Standard fixed-vocabulary protocol; utils/coco.txt is only a label list in this repository
Radford et al. [3], 2021	CLIP contrastive pre-training	400 million image–text pairs; zero-shot transfer over more than 30 datasets	Representation learning, not a real-time bounding-box detector
Liu et al. [4], 2024	Grounding DINO, ECCV 2024 published version	Open-set and referring-expression benchmarks under published protocols	Alternative open-set approach; not used in this repository
Cheng et al. [5], 2024	YOLO-World and RepVL-PAN	CVPR 2024, pp. 16901–16911	Direct model-family basis; published benchmark results are not reproduced here
Minderer et al. [11]; Li et al. [12]; Zhou et al. [13]; Wang et al. [14]	OWL-ViT, GLIP, Detic, YOLOE	ECCV/CVPR/arXiv protocols with distinct vocabularies and datasets	Expanded open-vocabulary context; incompatible with this repository's prompts and runtime
Alqahtani et al. [19], 2024	Edge accuracy–latency–energy benchmark	YOLOv8, SSD, and EfficientDet Lite across heterogeneous edge devices	Motivates separate timing instrumentation; its measurements are not reused as TRINEXIS results

B. Assistive Interfaces, Speech, and Privacy

Remote sighted assistance has been studied through Be My Eyes [16], while Seeing AI [17] and AI scene-description applications [18] illustrate practical auditory presentation of visual information. These systems are relevant to the design space but do not validate TRINEXIS. The repository configures 72 language codes and 62 top-level speech resources; because the translations were not produced or checked under a documented protocol, the title and text use “language-configured” rather than “multilingual.” Camera-based systems also raise privacy and face-recognition concerns [20], [21]. The optional face-recognition hook remains unavailable, and no consent, retention, or biometric-processing protocol is claimed.

C. Research Software and Reproducibility

Git LFS stores small pointer files in Git while large model content is held elsewhere [10]. The software is distributed as a tagged GitHub release [23] with CITATION.cff, codemeta.json, dependency-license notes, pinned manifests, and a weights download script that verifies Git LFS object identifiers as SHA-256 checksums. The project adopts AGPL-3.0-only [24] because Ultralytics YOLO is distributed under AGPL-3.0 [22]. A Zenodo DOI has not been minted; the release URL is cited instead. The source census and software checks are retained as appendix-level verification records.

III. SYSTEM DESIGN

A. Architecture and Control Flow

Figure 1 shows the runtime and control flow. A browser client calls Flask control routes and displays a Motion JPEG stream. A configurable camera backend supplies frames to OpenCV. Detector calls are gated by an interval throttle, after which YOLO-World produces text-conditioned detection records. Those records feed visual overlays, a relative distance index, a time-decayed heatmap, session telemetry, and an alert-aware speech selector. Optional face recognition is shown as a dashed, unavailable path. There is no training or benchmark-evaluation path.

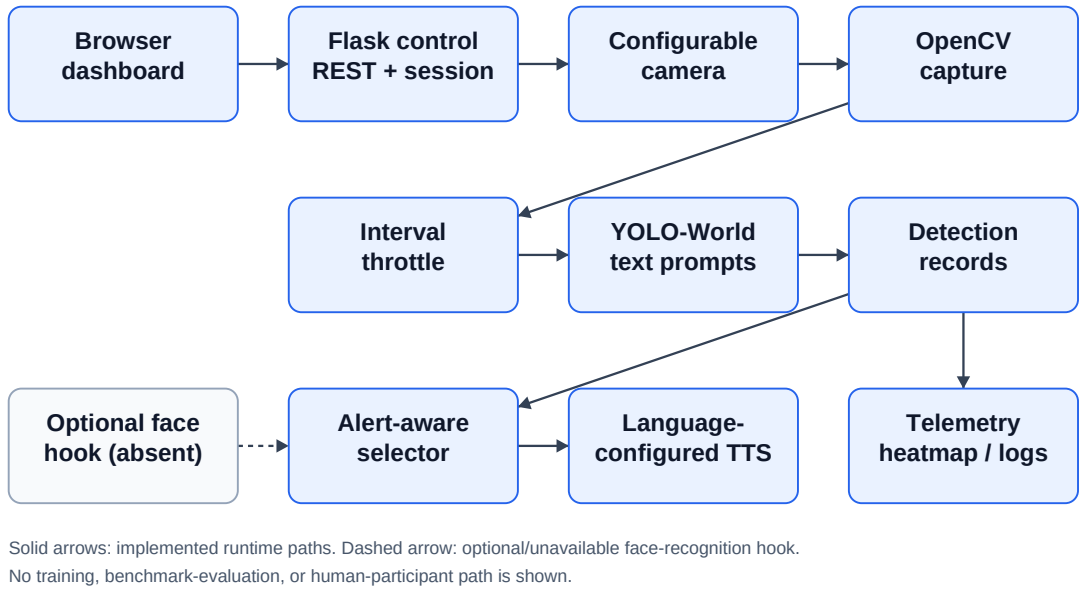

Fig. 1. TRINEXIS system architecture. Solid arrows denote implemented runtime paths; the dashed arrow denotes an optional/unavailable face-recognition hook. Vector SVG/PDF versions are provided in the release artifacts.

B. Interval Throttle and Alert-Aware Narration

Let t be the current capture time, t_prev the previous detector invocation, and Delta the configured throttle interval. A detector call is admitted when

```
t - t_prev >= Delta. (1)
```

The default is Delta = 1.5 s and the spoken shortlist is limited to two objects. Let A be the set of user-designated alert names, c(o) the detector confidence, and d(o) the relative distance index. The selector sorts every detection record o by the ascending key

```
R(o) = (I_alert(o), -c(o), d(o)), (2)
```

and only then takes the first two records. In Eq. (2), I_alert(o) = 0 when o.name is in A and 1 otherwise; alert objects therefore receive priority regardless of whether they are among the two highest-confidence detections. A confidence-first shortlist followed by alert-aware reranking inside the shortlist would not provide this invariant.

C. Time-Based Heatmap and Relative Distance Index

Eq. (3) replaces the older per-frame 0.995 multiplier, whose decay depends on camera frame rate, with elapsed wall-clock time dt and an adjustable half-life tau:

```
H_t = 2^(-dt/tau) * H_(t-1) + sum_i C(c_i; 35). (3)
```

In Eq. (3), H_t is the heatmap at time t, C(c_i; 35) is a filled circle of radius 35 pixels centered on detection box c_i, and tau defaults to 60 s. The 60 s value is an adjustable default chosen to retain visible activity for roughly one minute; it is not fitted to data. The distance overlay is a unitless, relative size-based index

$$d_idx = (W * F) / \max(w_px, 1), \quad (4)$$

where w_px is the observed box width, W = 60, and F = 360. W and F are heuristic constants, not calibrated physical measurements; d_idx has no metric unit and must not be interpreted as centimetres or metres.

D. Configuration, Security, and Deployment

Camera index, backend, resolution, host, port, optional API token, single-client mode, client timeout, session secret, and heatmap half-life are read from environment variables. The default bind address is 127.0.0.1 rather than 0.0.0.0. /set_model accepts only a checked-in model allowlist. Mutating control endpoints can require an X-API-Token or Bearer header, and single-client arbitration prevents a second browser from mutating shared state while the first session is active. Capture-frame and detector-invocation rates are reported separately.

TABLE II. IMPLEMENTATION FEATURES AND VERIFICATION

Feature	Implementation	Verification
Open-source distribution	AGPL-3.0 LICENSE [24], README, pinned requirements, CITATION.cff, codemeta.json, and dependency-license notes	Source-contract tests and README instructions
Confidence display	Dashboard reports AVG CONFIDENCE rather than ACCURACY	Source-contract test
Alert-aware speech	Rank alerts before shortlist truncation via trinexis_core.select_speech_objects	Unit test and seeded synthetic regression check
Client-state safety	Single-client control arbitration with stale-session timeout	Pure-function unit test
Network defaults	Default host 127.0.0.1; optional API token on mutating control routes	Source-contract and pure-function tests
Model loading	Allowlisted model filenames with HTTP 400 on rejection	Unit test
Heatmap decay	Wall-clock exponential decay with adjustable half-life	Unit test
Distance display	Unitless relative distance index with explicit non-calibration note	Unit and source-contract tests
Camera portability	Configurable camera index and backend	Unit and source-contract tests
Rate reporting	Separate capture_fps and inference_fps counters and dashboard displays	Source-contract test
Figures	Enlarged PNG plus standalone SVG and vector-PDF exports	make_updated_figures.py

IV. SOFTWARE VERIFICATION

A. Verification Environment

All checks were static or synthetic. The preparation environment used Python 3.11.2 with the standard library for tests; manuscript and figure generation used the pinned packages in paper/requirements-paper.txt. No model weights were hydrated, no camera was opened, no Flask server was run, no network speech request was made, and no human participant was involved. Figure 2 summarizes the workflow.

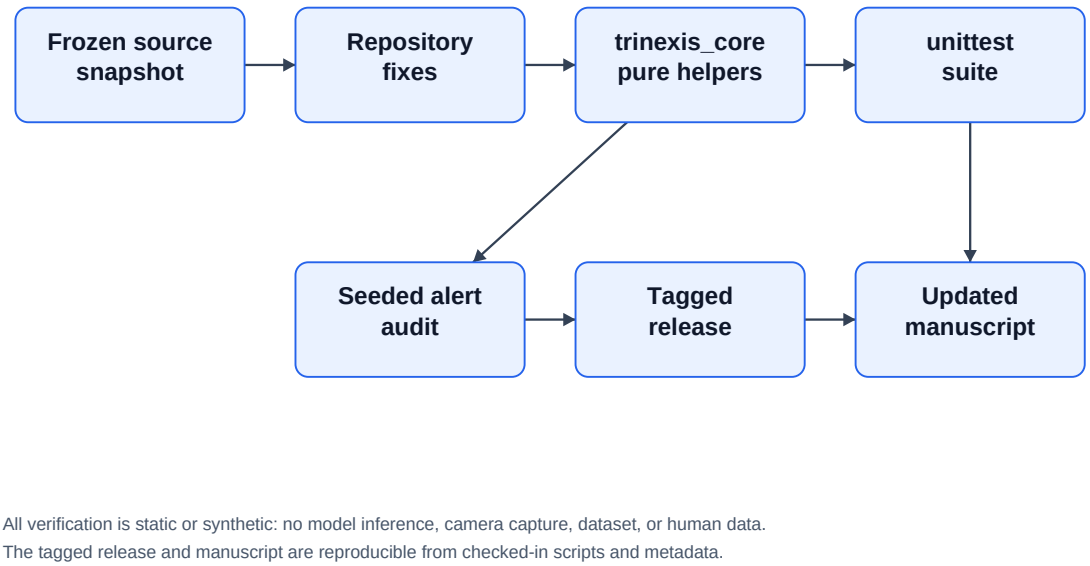

Fig. 2. Software-verification workflow. Every step is static or synthetic; no model inference, camera capture, labeled dataset, or human data is used. Vector SVG/PDF versions are provided in the release artifacts.

B. Alert-Selection Regression Check

The alert-selection check used 1,000 seeded synthetic detection cases, each with six records, one alert object, and a two-object speech shortlist. With six records and a top-2 shortlist, a uniformly ranked alert falls outside the shortlist with probability 4/6 (66.7%). The confidence-first ordering omitted the alert object in 681 of 1000 cases (68.1%), consistent with that probability setup; the alert-first ordering omitted it in 0 of 1000 cases (0.0%). This is a regression test of an ordering invariant. It does not show how often real detections contain alerts, whether a real user hears an alert, or whether the speech output is useful.

TABLE III. SEEDED SYNTHETIC ALERT-SELECTION REGRESSION CHECK

Metric	Confidence-first	Alert-first	Change
Cases with a configured alert object	1,000	1,000	—
Alert omissions from the two-object shortlist	681	0	−681
Omission rate	68.1%	0.0%	−68.1 pp
Evidence type	Synthetic ordering check	Synthetic ordering check	No model inference

C. Unit and Source-Contract Tests

The standard-library unittest suite contains 24 tests: 14 pure-policy tests and 10 source-contract tests. All 24 passed with zero failures and zero errors. The suite covers camera configuration, model allowlisting, alert-first ranking, confidence-first drop reproduction, time-based decay, relative distance index, rate calculation, token validation, single-client timeout, license/readme/requirements presence, corrected UI labels, configurable host/backend, and the absence of metric centimetre labels in the primary backend.

TABLE IV. TEST-SUITE SUMMARY

Test group	Tests	Failures	Errors
trinexis_core pure-policy tests	14	0	0
Repository/source-contract tests	10	0	0
Total	24	0	0

D. Release and Reproducibility

The software and manuscript are associated with GitHub release v0.3.0-standalone [23], CITATION.cff, and codemeta.json. CITATION.cff and codemeta.json provide machine-readable metadata; weights/download_weights.py parses the checked-in Git LFS pointers and verifies the declared SHA-256 object identifiers after Git LFS materialization. A Zenodo DOI has not been minted because author authorization is required; the release URL is cited instead. Appendix A lists reproduction commands, and Appendix B lists the principal artifacts.

E. Future Work

- Evaluate detection quality on a labeled dataset and ablate vocabulary size and confidence thresholds.
- Instrument capture, inference, speech, and rendering separately on declared CPU/GPU hardware.
- Calibrate the distance proxy at known distances or retain the unitless index with its current disclaimer.
- Obtain ethics approval for translated-template ratings and any assistive-use study.

V. DISCUSSION

A. Interpretation

TRINEXIS shows how an open-vocabulary detector can be wrapped in a bounded, testable interaction loop. Alert-first ranking makes the speech policy independent of whether an alert happens to be among the highest-confidence detections, while interval throttling separates detector cadence from camera frame rate. Time-based heatmap decay and the unitless distance index remove two frame-rate- and calibration-dependent ambiguities. The published detector literature is not comparable to this webcam pipeline because its datasets, prompts, splits, and hardware protocols differ.

B. Ethical and Practical Implications

Camera-based interfaces can expose bystanders and sensitive environments. Privacy, consent, retention, and biometric-processing policies must be defined before any real-world or assistive-use study [20], [21]. The AGPL-3.0 license [24] and dependency-license notice clarify redistribution obligations but do not substitute for an ethics review. The system should not be described as assistive technology for blind or low-vision users until a consented study demonstrates benefit.

VI. CONCLUSION AND FUTURE WORK

TRINEXIS is a documented, configurable, interval-throttled open-vocabulary webcam system with alert-aware, language-configured speech. Its software verification establishes the alert-ordering invariant and the behavior of the configuration, security, and display policies. The next steps are detection-quality evaluation, latency instrumentation, distance calibration or continued use of the unitless index, and ethics-approved speech/user studies. Those experiments would support a separate applied-vision or HCI workshop paper.

DECLARATIONS

A. Funding and Competing Interests

No external funding was reported for this work. No competing interests were disclosed by the author.

B. Author Contributions (CRediT)

Yug G. Lakhani: conceptualization, methodology, software, formal analysis, investigation, writing—original draft, and writing—review and editing. Validation and data curation are not listed because no validation dataset or curated research data were used.

C. AI-Tool Disclosure

Arena.ai Agent Mode was used for source auditing, figure generation, repository scripting, and manuscript formatting. The author is responsible for the final manuscript and for verifying every number and declaration against the target venue's policy.

D. Data and Code Availability

The software and manuscript are available from GitHub release v0.3.0-standalone [23] under AGPL-3.0-only [24]. Ultralytics' AGPL-3.0 license is cited directly [22]. Model weights are Git LFS pointers and are fetched and checked with weights/download_weights.py [10]. No labeled dataset, experimental log, or valid Zenodo DOI is claimed. Dependency licenses are documented in THIRD_PARTY_LICENSES.md.

REFERENCES

[1] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You Only Look Once: Unified, Real-Time Object Detection,” in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), 2016, pp. 779–788, doi: 10.1109/CVPR.2016.91.

[2] T.-Y. Lin et al., “Microsoft COCO: Common Objects in Context,” in Proc. Eur. Conf. Comput. Vis. (ECCV), 2014, pp. 740–755.

[3] A. Radford et al., “Learning Transferable Visual Models From Natural Language Supervision,” in Proc. Int. Conf. Mach. Learn. (ICML), PMLR 139, 2021, pp. 8748–8763.

[4] S. Liu et al., “Grounding DINO: Marrying DINO with Grounded Pre-Training for Open-Set Object Detection,” in Computer Vision – ECCV 2024, LNCS vol. 15105, 2024, pp. 38–55, doi: 10.1007/978-3-031-72970-6_3.

[5] T. Cheng, L. Song, Y. Ge, W. Liu, X. Wang, and Y. Shan, “YOLO-World: Real-Time Open-Vocabulary Object Detection,” in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), 2024, pp. 16901–16911, doi: 10.1109/CVPR52733.2024.01599.

[6] Ultralytics, “YOLO-World documentation,” accessed Nov. 10, 2026. [Online]. Available: https://docs.ultralytics.com/models/yolo-world/

[7] Flask Project, “Flask documentation,” accessed Nov. 10, 2026. [Online]. Available: https://flask.palletsprojects.com/

[8] OpenCV, “Getting Started with Videos,” accessed Nov. 10, 2026. [Online]. Available: https://docs.opencv.org/4.13.0/dd/d43/tutorial_py_video_display.html

[9] P. N. Durette, “gTTS: Python library and CLI tool to interface with Google Translate's text-to-speech API,” GitHub repository, accessed Nov. 10, 2026. [Online]. Available: https://github.com/pndurette/gTTS

[10] Git LFS Project, “Git LFS Specification: The Pointer,” accessed Nov. 10, 2026. [Online]. Available: https://github.com/git-lfs/git-lfs/blob/main/docs/spec.md

[11] M. Minderer et al., “Simple Open-Vocabulary Object Detection,” in Computer Vision – ECCV 2022, LNCS, 2022, pp. 728–755, doi: 10.1007/978-3-031-20080-9_42.

[12] L. H. Li et al., “Grounded Language-Image Pre-Training,” in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), 2022, pp. 10955–10965, doi: 10.1109/CVPR52688.2022.01069.

[13] X. Zhou, R. Girdhar, A. Joulin, P. Krähenbühl, and I. Misra, “Detecting Twenty-Thousand Classes Using Image-Level Supervision,” in Computer Vision – ECCV 2022, LNCS vol. 13669, 2022, pp. 350–368, doi: 10.1007/978-3-031-20077-9_21.

[14] A. Wang, L. Liu, H. Chen, Z. Lin, J. Han, and G. Ding, “YOLOE: Real-Time Seeing Anything,” arXiv:2503.07465, 2025, doi: 10.48550/arXiv.2503.07465.

[15] Ultralytics, “Ultralytics YOLO,” GitHub repository, accessed Nov. 10, 2026. [Online]. Available: https://github.com/ultralytics/ultralytics

[16] M. Avila, K. Wolf, A. Brock, and N. Henze, “Remote Assistance for Blind Users in Daily Life: A Survey about Be My Eyes,” in Proc. 9th ACM Int. Conf. Pervasive Technol. Related Assistive Environ. (PETRA), 2016, Article 85, 2 pages, doi: 10.1145/2910674.2935839.

[17] Microsoft, “Seeing AI,” accessed Nov. 10, 2026. [Online]. Available: https://www.seeingai.com/

[18] R. E. Gonzalez Penuela, J. Collins, C. L. Bennett, and S. Azenkot, “Investigating Use Cases of AI-Powered Scene Description Applications for Blind and Low Vision People,” in Proc. CHI Conf. Human Factors Comput. Syst. (CHI), 2024, Article 901, pp. 1–21, doi: 10.1145/3613904.3642211.

[19] D. K. Alqahtani, M. A. Cheema, M. A. Rodriguez, and A. N. Toosi, “A Comprehensive Evaluation of Deep Learning Object Detection Models on Heterogeneous Edge Devices,” arXiv:2409.16808, v1 2024, revised 2026, doi: 10.48550/arXiv.2409.16808.

[20] I. D. Raji, T. Gebru, M. Mitchell, J. Buolamwini, J. Lee, and R. Denton, “Saving Face: Investigating the Ethical Concerns of Facial Recognition Auditing,” in Proc. AAAI/ACM Conf. AI, Ethics, Soc. (AIES), 2020, pp. 145–151, doi: 10.1145/3375627.3375820.

[21] M. R. Hasan, R. Guest, and F. Deravi, “Presentation-level Privacy Protection Techniques for Automated Face Recognition—A Survey,” ACM Comput. Surv., vol. 55, no. 13s, Article 286, pp. 1–27, 2023, doi: 10.1145/3583135.

[22] Ultralytics, “LICENSE,” GitHub repository, AGPL-3.0, accessed Nov. 10, 2026. [Online]. Available: https://github.com/ultralytics/ultralytics/blob/main/LICENSE

[23] Unusualyug, “tech_yolo release v0.3.0-standalone,” GitHub, 2026. [Online]. Available: https://github.com/Unusualyug/tech_yolo/releases/tag/v0.3.0-standalone

[24] Free Software Foundation, “GNU Affero General Public License, Version 3,” 2007. [Online]. Available: https://www.gnu.org/licenses/agpl-3.0.txt

APPENDIX A. REPRODUCTION COMMANDS

```
git clone https://github.com/Unusualyug/tech_yolo.git
cd tech_yolo
git checkout v0.3.0-standalone
python3 -m pip install -r requirements.txt
python3 weights/download_weights.py --fetch --verify
python3 experiments/alert_selection_audit.py
python3 -m unittest discover -s tests -v
python3 -m pip install -r paper/requirements-paper.txt
python3 paper/make_updated_figures.py
python3 paper/build_updated_manuscript.py
```

APPENDIX B. PRINCIPAL ARTIFACTS

- paper/tech_yolo_updated_research_paper.pdf, .docx, and .md: standalone technical report.
- paper/artifacts/alert_selection_audit.json: seeded synthetic alert-selection regression check.
- paper/artifacts/fig1_system_architecture_updated.{png,svg,pdf} and fig2_implementation_verification.{png,svg,pdf}: figure exports.
- paper/artifacts/updated_repository_audit.json: machine-readable release/test summary.
- weights/download_weights.py and weights/weights_checksums.sha256: Git LFS pointer parser and SHA-256 manifest.
- zenodo.json and ZENODO.md: prepared Zenodo metadata and DOI-minting instructions.
- paper/reference_verification.md: per-reference DOI/publisher verification log.
- trinexis_core.py, tests/, experiments/, CITATION.cff, codemeta.json, README.md, LICENSE, requirements.txt, and THIRD_PARTY_LICENSES.md: software artifacts.